

AstroPilot — Project Context

1. Vision produit

AstroPilot est un moteur d'aide à la décision pour l'astrophotographie.

Sa question centrale est :

Que puis-je photographier ce soir avec mon matériel, depuis ma position, et quelle mission maximise réellement la valeur de ma nuit ?

L'objectif final est une application Android/iOS reposant sur un moteur Python, puis une API FastAPI.

Architecture cible :

AstroPilot Engine (Python) → FastAPI → Android / iOS

Domaine officiel : <https://astropilot.io>

Dépôt GitHub : <https://github.com/touchthebitum/astropilot>

2. État du projet au 21 août 2026

Branche de développement actuelle : `feature-opportunity-engine`

État Git au dernier contrôle :

- branche en avance de 174 commits sur `origin/feature-opportunity-engine`
- working tree propre
- `astro_score.py` : 1846 lignes
- 82 tests collectés et passants
- modes CLI validés : `tonight`, `portfolio`, `calendar`, `full`

Le refactoring majeur de l'ancien moteur monolithique est désormais très avancé.

3. Pipeline décisionnel actuel

Le pipeline principal est désormais :

Forecast → NightEvaluation → Portfolio enrichment → Candidate generation → OpportunityEngine → RecommendationEngine → OpportunityRecommendationService → TonightRunner → TonightMissionService → NightMission → MissionPresenter

Le moteur ne se contente plus d'attribuer un score astronomique.

Il combine désormais :

- qualité astronomique de la cible
- météo
- Lune
- cadrage

- résolution
- sampling
- matériel disponible
- état du portefeuille
- priorité projet
- ROI de session
- progression marginale
- bonus de clôture
- diversification
- risque de report
- opportunités futures
- productivité de nuit
- saison

4. Forecast

Composants principaux :

- `decision/forecast/forecast_engine.py`
- `decision/forecast/night_evaluation.py`

`ForecastEngine` construit les évaluations de nuit à partir des données météo, des heures astronomiques, de la Lune, du catalogue, des règles de décision et de l'évaluation de chaque cible.

`NightEvaluation` contient notamment :

- `all_results`
- `top3`
- `best_score`
- `best_object`
- `best`
- `setup_name`
- `night_score`
- `top_objects_for_night`

`PortfolioEngine` ne recalcule plus l'état projet. Sa responsabilité actuelle est uniquement de garantir que les projets du portefeuille restent présents dans `top_objects_for_night`, même lorsqu'ils ne font pas partie des meilleures cibles astronomiques brutes.

5. Portfolio

Composants principaux :

- `decision/portfolio/project_state.py`
- `decision/portfolio/project_scoring.py`
- `decision/portfolio/project_gain.py`
- `decision/portfolio/diversification.py`
- `decision/portfolio/candidate_scoring.py`
- `decision/portfolio/portfolio_engine.py`

- `decision/portfolio/portfolio_forecast_engine.py`
- `decision/portfolio/portfolio_presenter.py`

État projet

Le moteur suit notamment :

- heures acquises
- heures cibles
- progression
- heures restantes
- importance projet

Priority

`project_priority()` normalise l'importance projet sur une échelle 0–100 : **importance × 10**.

ROI

L'ancien `project_roi()` global a été supprimé. Le moteur utilise désormais principalement un ROI lié à la session et à l'opportunité réelle d'acquisition.

Bonus de clôture

Le bonus de clôture est désormais basé sur les heures restantes et la capacité réelle de la session. La logique a été unifiée avec `closure_bonus_for_remaining()`.

Score simulé de portefeuille

Le score de roadmap dynamique repose actuellement sur :

- importance projet pondérée
- bonus de clôture
- opportunité future

Le biais artificiel lié à la progression cumulée a été supprimé. L'importance a été rééquilibrée avec un facteur ×6 dans le scoring simulé afin d'éviter qu'elle écrase complètement l'urgence/opportunité.

6. PortfolioForecastEngine

`PortfolioForecastEngine` est désormais la source canonique pour la planification multi-nuits.

L'ancien système parallèle de classement portefeuille, calendrier fixe, forecast de complétion et roadmap legacy a été supprimé.

Le moteur travaille sur une copie simulée du portefeuille. À chaque étape, il :

1. calcule les projets encore actifs
2. calcule les heures restantes simulées
3. demande à `FutureOpportunityEngine` une estimation basée sur cet état simulé
4. calcule le score portefeuille
5. ajoute le bonus d'opportunité
6. sélectionne le meilleur projet
7. applique virtuellement les heures disponibles

8. recommence jusqu'à épuisement de la capacité connue

Le `FutureOpportunityEngine` accepte désormais `remaining_hours=...` afin de ne plus se baser uniquement sur l'état réel persistant du projet pendant une simulation.

Ceci corrigeait un défaut important : auparavant, la roadmap simulée continuait à raisonner comme si les heures virtuellement acquises n'existaient pas.

7. Opportunity et Recommendation

Composants principaux :

- `decision/opportunity/opportunity_engine.py`
- `decision/opportunity/opportunity.py`
- `decision/opportunity/action.py`
- `decision/opportunity/explanation.py`
- `decision/recommendation/recommendation_engine.py`
- `decision/recommendation/recommendation.py`
- `decision/services/opportunity_recommendation_service.py`

Le pipeline Opportunity/Recommendation est désormais la couche canonique entre les candidats portefeuille et la mission finale.

Objectif architectural : séparer clairement évaluation, opportunité, recommandation, mission et présentation.

8. Sélection de projet pour la nuit

La sélection active combine notamment :

- score astronomique
- pondération utilisateur astro/projet
- priorité projet
- ROI de session
- bonus de progression marginale
- bonus de clôture
- diversification
- risque de report
- opportunité future

`NightStrategyEngine` permet de produire plusieurs stratégies : balanced, roi, completion, diversification, risk.

Le fallback reste le score global calculé par le pipeline historique modernisé.

9. FutureOpportunityEngine

Composants :

- `decision/engines/future_opportunity_engine.py`
- `decision/models/future_opportunity.py`

Le moteur estime notamment :

- nombre de bonnes nuits restantes
- ratio météo
- nuits nécessaires
- ratio d'opportunité
- niveau de risque

Le moteur peut travailler avec l'état réel du portefeuille ou un nombre d'heures restantes simulé. Cette seconde possibilité est essentielle pour les roadmaps dynamiques.

10. Risque de report

Composants :

- `decision/risk/postponement_impact.py`
- `decision/risk/project_risk_context.py`
- `decision/risk/project_risk_context_builder.py`
- `decision/risk/risk_engine.py`
- `decision/risk/risk_report.py`

Le risque de report tient compte notamment de l'urgence, du nombre de nuits nécessaires, de la pression stratégique, de la priorité projet, de la confiance et de la qualité astronomique.

11. Productivité de nuit

Deux couches existent encore :

- `decision/productivity/`
- `decision/night_productivity/`

Le moteur actuel calcule notamment :

- heures astronomiques
- heures réellement productives
- confiance de productivité
- fenêtres exploitables
- slices temporelles

Cette partie devra encore être clarifiée architecturalement à terme afin d'éviter une duplication conceptuelle entre `productivity` et `night_productivity`.

12. Intelligence Layer

Composants :

- `decision/intelligence/analysis_context.py`
- `decision/intelligence/analysis_result.py`
- `decision/intelligence/season_analysis.py`

`SeasonAnalysis` constitue la première couche d'intelligence structurée indépendante.

Pipeline : **SeasonEngine** → **SeasonAnalysis** → **AnalysisResult** → **NightMission** → **MissionPresenter**

Les futurs moteurs d'intelligence devront utiliser des structures communes plutôt que du texte construit directement dans le moteur principal.

13. Saison

Composants :

- `decision/season/season_engine.py`
- `decision/season/dynamic_season_engine.py`
- `decision/season/season_data.py`

La saison doit à terme être calculée dynamiquement à partir de RA, DEC, latitude observateur, date, durée de nuit et altitude utile.

L'objectif reste de supprimer toute dépendance inutile à des fenêtres saisonnières statiques.

14. Mission

Composants principaux :

- `decision/mission/mission_input.py`
- `decision/mission/mission_builder.py`
- `decision/mission/mission_assembler.py`
- `decision/mission/night_mission.py`
- `decision/mission/night_planner.py`
- `decision/mission/timeline_builder.py`
- `decision/mission/equipment_builder.py`
- `decision/mission/mission_presenter.py`
- `decision/services/tonight_mission_service.py`

La mission finale contient notamment :

- cible recommandée
- fenêtre optimale
- durée conseillée
- gain attendu
- météo
- Lune
- matériel
- filtre
- timeline de nuit
- analyse saison
- productivité
- risque de report
- explications de recommandation

15. Matériel

Les modèles matériels typés existent dans `decision/models/equipment/`.

Ils couvrent notamment :

- caméra
- optique
- monture
- filtres
- roue à filtres
- autofocus
- guidage
- rotateur
- accessoires
- capacités du setup

La sélection matérielle actuelle repose sur `select_best_setup_for_object()`.

Les commandes CLI `--object` et `--target-object` réutilisent ce moteur canonique.

Le flag CLI `--compare`, devenu inutilisé, a été supprimé.

16. CLI actuelle

Modes principaux :

- `--mode tonight`
- `--mode portfolio`
- `--mode calendar`
- `--mode full`

Autres options :

- `--equipment`
- `--goal`
- `--object`
- `--target-object`

tonight

Produit la mission recommandée pour la nuit et l'état prévu du portefeuille.

portfolio

Produit la projection dynamique du portefeuille.

calendar

Affiche la roadmap multi-nuits dynamique.

full

Affiche à la fois la roadmap dynamique, la couverture du portefeuille et l'état prévu en fin d'horizon.

Le mode `calendar` et le mode `portfolio` n'utilisent plus l'ancien moteur calendrier legacy.

17. Tests

État validé au 21 août 2026 : **82 tests passants**.

Principaux contrats architecturaux testés :

- Candidate score
- coordonnées et unités
- contexte décisionnel
- état projet
- FutureOpportunity avec état simulé
- sécurité des imports
- MissionInput
- MissionPresenter
- stratégies de nuit
- OpportunityEngine
- scoring portefeuille
- diversification
- risque de report
- productivité
- gain projet
- scoring projet
- Season bridge
- persistance du profil utilisateur

Les quatre runtimes ont également été validés :

- tonight : OK
- portfolio : OK
- calendar : OK
- full : OK

18. État de `astro_score.py`

`astro_score.py` reste encore le principal point d'intégration du système.

Taille actuelle : **1846 lignes**.

Le fichier contenait auparavant plus de 4700 lignes.

Le refactoring a supprimé ou extrait une grande quantité de logique legacy.

Les fonctions restantes ont été auditées :

- aucun import inutilisé
- aucune fonction réellement orpheline détectée
- plusieurs fonctions sont conservées comme callbacks injectés dans les nouveaux moteurs

La priorité n'est désormais plus de réduire artificiellement la taille du fichier, mais de déplacer uniquement les responsabilités dont les frontières architecturales sont claires.

19. Sprint feature-opportunity-engine — bilan

Le sprint a notamment réalisé :

- migration du mode portfolio vers `PortfolioForecastEngine`
- migration du calendrier vers la roadmap dynamique
- migration du mode full vers la roadmap dynamique
- suppression du calendrier legacy
- suppression du classement portefeuille legacy
- suppression du forecast de complétion legacy
- suppression de `project_roi`
- suppression du ROI transporté inutilement dans les évaluations objet
- suppression du ROI candidat devenu redondant
- adoption du ROI de session pour le scoring candidat
- normalisation de la priorité projet
- suppression de plusieurs doubles pondérations
- utilisation des heures restantes simulées dans `FutureOpportunityEngine`
- suppression du biais de progression dans le score simulé
- unification du bonus de clôture
- rééquilibrage de l'importance projet
- simplification de `PortfolioEngine`
- suppression de nombreux helpers morts
- suppression des imports inutilisés
- suppression du flag CLI `--compare`
- extraction de l'analyse CLI `--target-object`
- validation finale des quatre modes CLI

Commits récents représentatifs :

- `ec74d2c` refactor: extract target object cli analysis
- `80d54e6` cleanup: remove unused compare cli flag
- `8d2f65f` cleanup: remove unused imaging score helper
- `2377507` cleanup: remove unused astro score imports
- `11f3e08` cleanup: remove unused project state fields
- `0ae58a0` cleanup: remove redundant portfolio enrichment
- `3d03cc4` cleanup: remove unused forecast capacity helper
- `fa48d96` cleanup: remove unused project roi
- `a3f1d54` refactor: migrate calendar modes to dynamic forecast
- `6fcaa7a` refactor: migrate portfolio mode to dynamic forecast
- `d728078` fix: rebalance importance in portfolio forecast
- `f1a1c50` refactor: unify portfolio closure scoring
- `5bf9e47` fix: remove progress bias from portfolio forecast score
- `57599d0` fix: use simulated remaining hours in portfolio forecast

20. Principes de développement

Pour chaque changement architectural sensible :

1. audit des références
2. modification minimale
3. `python -m py_compile`
4. `git diff --check`
5. `pytest -q`
6. exécution du mode runtime concerné
7. inspection du diff
8. commit isolé

Ne pas supprimer une fonction sur la seule base d'un outil statique.

Toujours vérifier appels directs, callbacks, injection de dépendances, tests et chemins CLI.

Avant chaque commit important :

- supprimer les prints de debug temporaires
- supprimer les logs temporaires
- vérifier `git status --short`

Après chaque sprint majeur ou décision architecturale importante :

- mettre à jour `docs/PROJECT_CONTEXT.md`
- mettre à jour la version PDF de PROJECT_CONTEXT si elle est maintenue

21. Roadmap produit

Astro Quality Index — AQI

Construire un indice de qualité réellement astrophotographique tenant compte notamment :

- couches nuageuses
- cirrus
- transparence
- humidité
- risque de rosée
- Lune
- vent
- seeing
- focale
- sampling
- filtre
- cible
- matériel

L'AQI doit produire un score spécifique au couple **cible + setup + filtre + site + heure**, et non une météo générique.

Guardian Mode

Mode destiné aux sessions non surveillées.

Il devra surveiller notamment :

- pluie
- vent
- humidité
- rosée
- risque matériel
- autres conditions critiques

Les alertes devront être graduées : information silencieuse, avertissement, alerte urgente, alarme critique en cas de danger matériel.

Opportunity Alerts

Système d'alertes configurable permettant de signaler une opportunité réellement intéressante selon météo, qualité, cible, filtre, site, équipement, portefeuille et score minimum.

Les alertes doivent rester silencieuses lorsque l'opportunité n'est pas suffisamment forte.

Mode collaboratif

Permettre à plusieurs astrophotographes disposant de matériel compatible de travailler sur un même projet.

Le système devra :

- cumuler le temps d'intégration
- suivre les contributions
- suivre le temps par filtre
- identifier les filtres manquants
- recommander qui doit capturer quoi
- maximiser la valeur du dataset collectif

Mode nuit

Prévoir un affichage dédié à l'utilisation nocturne :

- faible luminosité
- couleurs compatibles vision nocturne
- interface simplifiée
- limitation des éléments éblouissants

Mobile / API

Étapes futures :

- FastAPI
- authentification
- gestion utilisateurs
- GPS
- timezone automatique
- synchronisation cloud

- Android
- iOS
- notifications

22. Dette technique restante

Principaux points à surveiller :

- `astro_score.py` reste encore un fichier d'intégration conséquent
- certains sous-systèmes historiques et modernes coexistent encore
- clarification future entre `productivity` et `night_productivity`
- extraction progressive des presenters CLI
- migration éventuelle de la sélection matériel vers un service dédié
- saison dynamique à finaliser
- catalogue à enrichir
- persistance utilisateur à consolider
- API non encore implémentée

Le nettoyage ne doit plus être poursuivi uniquement pour réduire le nombre de lignes.

Chaque extraction future doit résoudre une responsabilité architecturale réelle.

23. Règle de reprise du projet

Lors d'une nouvelle session de développement :

1. lire ce fichier
2. vérifier la branche active
3. exécuter `git status --short`
4. vérifier les derniers commits
5. lancer `pytest -q`
6. vérifier la taille de `astro_score.py`
7. ne pas supposer que les anciens chemins legacy existent encore
8. poursuivre à partir des moteurs actuels